Architecture et Systèmes d'Exploitation : Gestion des Processus et Signaux POSIX

Thème principal : Gestion des Processus, Cycles de Vie, Multitâche Préemptif et Communication Inter-Processus par Signaux POSIX

Synthèse exécutive : Ce cours aborde les mécanismes fondamentaux de la gestion des tâches au sein des systèmes d'exploitation de type Unix/Linux. Il détaille la transition critique entre un programme statique stocké sur disque et un processus dynamique résidant en mémoire RAM. Pour permettre l'exécution simultanée de multiples applications sur un même ordinateur, le noyau (kernel) s'appuie sur le multitâche préemptif et un modèle d'états à trois niveaux (Élu, Prêt, Bloqué). La séance explicite également l'arborescence généalogique des processus (PID/PPID) issue du processus initial systemd/init (PID 1), le fonctionnement des appels système fork() et exec(), ainsi que le rôle des signaux POSIX (SIGINT, SIGTERM, SIGKILL, SIGSTOP, SIGCONT) pour le contrôle et la communication inter-processus.

Du Programme Statique au Processus Dynamique : Cycle de Vie, Attributs et Multitâche

Pour exécuter un code informatique, le système d'exploitation doit faire basculer le programme (données passives sur disque) vers l'état de processus (entité active en mémoire RAM). L'ordonnanceur du noyau orchestre cette activité via un modèle à trois états fondamentaux. Lorsqu'un processus est à l'état 'Élu', il occupe physiquement le processeur. Si son quantum de temps CPU expire, le noyau le bascule à l'état 'Prêt' et attribue le CPU à un autre processus. Si le processus déclenche une demande de lecture sur disque ou attend une saisie clavier, il bascule à l'état 'Bloqué'. Dès que la donnée est disponible, le noyau ne le remet pas immédiatement sur le CPU, mais le repasse à 'Prêt' dans la file d'attente de l'ordonnanceur. Tous les processus sous Unix dérivent d'un arbre généalogique strict démarrant au processus racine PID 1 (systemd/init). Lors de la création d'un processus, l'appel système fork() crée une copie conforme du processus père, puis exec() charge le nouveau code binaire dans le bloc mémoire de l'enfant.

- • Programme vs Processus : Un programme est un fichier exécutable passif stocké sur le disque dur ; un processus est une instance dynamique en cours d'exécution occupant de la mémoire RAM et des ressources processeur.
- • Multitâche préemptif : Mécanisme par lequel l'ordonnanceur (scheduler) du noyau alloue de très courtes tranches de temps CPU (quanta) à chaque processus pour simuler une exécution simultanée.
- • Bloc de Contrôle du Processus (PCB) : Structure de données interne au noyau regroupant toutes les informations d'un processus (PID, PPID, état, registres CPU, carte mémoire, identifiant UID, descripteurs de fichiers ouverts).
- • Arborescence PID/PPID : Chaque processus possède un identifiant unique (PID - Process ID) et pointe vers le PID de son créateur (PPID - Parent Process ID). Le processus racine utilisateur est systemd ou init (PID 1).
- • Appels système fork() et exec() : fork() duplique le processus parent à l'identique pour créer un enfant, puis exec() remplace le code exécutable de l'enfant par un nouveau programme.
- • Trois états d'un processus : Élu (Running - exécute son code sur le CPU), Prêt (Ready - attend qu'un quantum CPU lui soit attribué), Bloqué (Waiting - suspendu en attente d'une opération d'E/S ou d'un événement externe).

Cas concret enseignant : *Analogie du livre de cuisine : Le programme sur disque dur est comparé à un livre de recettes sagement rangé dans une bibliothèque (statique, passif, ne consommant ni énergie ni ingrédients). Le processus est le cuisinier en train d'exécuter activement la recette dans sa cuisine (dynamique, consommant de l'espace sur le plan de travail/RAM, du temps de cuisson/CPU et manipulant des ustensiles/descripteurs de fichiers).*

Contrôle et Communication Inter-Processus : Les Signaux POSIX

Les signaux représentent la forme de communication inter-processus (IPC) la plus élémentaire et asynchrone dans les systèmes POSIX. Lorsqu'un signal est émis à destination d'un processus, le noyau interrompt le fil d'exécution

normal du programme pour exécuter le gestionnaire associé. Un développeur a la possibilité de programmer son application pour intercepter (catch) ou ignorer (ignore) la plupart des signaux. Par exemple, un éditeur de texte peut intercepter SIGINT ou SIGTERM pour proposer la sauvegarde automatique du document avant de quitter. Cependant, la norme POSIX garantit l'autorité absolue de l'administrateur système en rendant deux signaux strictement incapturables et inignorables : SIGKILL (9) et SIGSTOP (19). Ces deux signaux sont traités en direct par le noyau, permettant d'arrêter impérativement tout processus verrouillé dans une boucle infinie ou consommant abusivement des ressources.

- Signal POSIX : Interruption logicielle asynchrone transmise à un processus pour l'informer d'un événement ou lui imposer une instruction.
- Gestionnaire de signal (Signal Handler) : Fonction spécifique exécutée par un processus lorsqu'il reçoit un signal pour en personnaliser le comportement (interception / catch).
- SIGINT (Signal 2) : Émis via 'Ctrl + C' depuis le terminal ; demande l'interruption douce d'une tâche interactive.
- SIGTERM (Signal 15) : Signal de terminaison envoyé par défaut par la commande 'kill'. Il invite le processus à libérer ses ressources, sauvegarder ses données et s'arrêter proprement.
- SIGKILL (Signal 9) : Ordre d'extinction inconditionnel géré directement par le noyau Linux. Il détruit immédiatement le processus sans lui laisser la possibilité d'intercepter ou d'ignorer le signal.
- Job Control (SIGSTOP 19 / SIGCONT 18) : 'Ctrl + Z' envoie SIGSTOP pour suspendre un processus ; les commandes 'fg' (foreground) ou 'bg' (background) envoient SIGCONT pour reprendre son exécution.

Cas concret enseignant : *Analogie de l'expulsion de l'immeuble : SIGTERM (15) est le gardien d'immeuble qui frappe poliment à la porte pour vous demander de quitter les lieux (vous avez le temps de ranger vos affaires, sauvegarder vos fichiers et fermer la porte à clé). SIGKILL (9) est une intervention d'urgence qui défonce la porte et vous dégage sur-le-champ sans vous laisser la moindre seconde pour toucher à quoi que ce soit.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi la commande 'kill 9 1234' renvoie-t-elle une erreur dans le terminal alors que 'kill -9 1234' fonctionne parfaitement ?
Réponse du professeur : Le tiret '-' devant le nombre indique à la commande kill qu'il s'agit du numéro de signal à transmettre (ici le signal 9, SIGKILL). Si vous omettez le tiret et tapez 'kill 9 1234', la commande interprète '9' comme un identifiant de processus (PID) ! Elle essaie donc d'envoyer le signal par défaut (SIGTERM 15) au processus PID 9 ainsi qu'au processus PID 1234. Si le PID 9 n'existe pas ou n'appartient pas à votre session utilisateur, le système retourne une erreur de permission ou de processus introuvable.

Question étudiant : Qu'est-ce qu'un processus dit 'Zombie' et comment peut-on le détruire à l'aide de la commande kill ?
Réponse du professeur : Un processus Zombie est un processus enfant qui a terminé son exécution (il a libéré sa mémoire RAM et ses ressources CPU) mais dont le processus parent n'a pas encore lu le code de sortie via l'appel système wait(). Il ne subsiste sous forme de 'zombie' que comme une ligne résiduelle dans la table des processus du noyau. On NE PEUT PAS tuer un processus Zombie avec 'kill -9' car il est DÉJÀ mort ! Pour le faire disparaître de la table des processus, il faut forcer le processus père à lire son état, ou tuer le processus père : le Zombie devient alors 'orphelin' et est immédiatement adopté puis nettoyé par le processus racine systemd/init (PID 1).

■ Exercice & Démonstration d'Application

Énoncé : Dans un terminal Bash Linux, un étudiant saisit la séquence de commandes suivante : 1. sleep 600 & 2. ps -ef | grep sleep 3. kill -15 Explicitez le rôle du symbole '&', détaillez le traitement réalisé lors de l'envoi du signal -15 et décrivez le résultat de la vérification par la commande ps.

Résolution pas à pas :
Étape 1 : Le symbole '&' placé en fin de ligne indique au shell d'exécuter la commande 'sleep 600' en arrière-plan (background). Le shell affiche l'identifiant du job ainsi que le PID attribué par le système (ex: [1] 4521) et rend immédiatement l'invite de commande à l'utilisateur. Étape 2 : La commande 'ps -ef | grep sleep' liste l'ensemble des

processus du système et filtre la sortie pour repérer le processus 'sleep'. Elle permet de vérifier son PID (4521) et son PPID (qui correspond au PID du shell Bash hôte). Étape 3 : La commande 'kill -15 4521' transmet le signal SIGTERM (Signal 15) au processus. Comme l'utilitaire 'sleep' ne possède pas de gestionnaire personnalisé interceptant ce signal, l'action par défaut du système s'applique : le processus interrompt proprement son exécution et se termine.

Résultat : Le processus sleep est définitivement arrêté. Une nouvelle exécution de 'ps -ef | grep sleep' confirme sa disparition totale de la table des processus actifs.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le nom et le PID du tout premier processus utilisateur démarré par le noyau Linux au lancement de la machine ?

R : Le premier processus utilisateur est nommé 'systemd' (ou 'init' sur les anciennes distributions Unix/Linux) et porte obligatoirement le PID 1.

Q : Quel signal POSIX est transmis au processus s'exécutant au premier plan lorsqu'un utilisateur presse la combinaison 'Ctrl + C' dans son terminal ?

R : La combinaison 'Ctrl + C' génère et transmet le signal SIGINT (Signal numéro 2).

Q : Pourquoi le signal SIGKILL (Signal 9) ne peut-il jamais être intercepté ni bloqué par un programme informatique ?

R : Parce que le signal SIGKILL n'est pas transmis au gestionnaire de signal de l'application : il est intercepté et exécuté directement par le noyau Linux lui-même, garantissant à l'administrateur de pouvoir stopper un processus bloqué ou malveillant.